

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Факультет информационных технологий и прикладной математики
Кафедра вычислительной математики и программирования

**Лабораторная работа №4 по курсу
«Операционные системы»**

Студент: Черноус Алексей Тимофеевич
Группа: М8О-209Б-23
Вариант: 21
Преподаватель: Миронов Евгений Сергеевич
Оценка: _____
Дата: _____
Подпись: _____

Москва, 2024

Содержание

1. Репозиторий
2. Постановка задачи
3. Общие сведения о программе
4. Общий метод и алгоритм решения
5. Исходный код
6. Сборка программы
7. Демонстрация работы программы
8. Выводы

Репозиторий

https://github.com/HGRaicer/MAI_OS/tree/main/lab4

Постановка задачи

Цель работы

Приобретение практических навыков в:

- Создание динамических библиотек
- Создание программ, которые используют функции динамических библиотек

Задание

Требуется создать динамические библиотеки, которые реализуют определенный функционал.

Далее использовать данные библиотеки 2-мя способами:

1. Во время компиляции (на этапе «линковки»/linking)
2. Во время исполнения программы. Библиотеки загружаются в память с помощью

интерфейса ОС для работы с динамическими библиотеками

В конечном итоге, в лабораторной работе необходимо получить следующие части:

- Динамические библиотеки, реализующие контракты, которые заданы вариантом;
- Тестовая программа (программа No1), которая использует одну из библиотек, используя знания полученные на этапе компиляции;
- Тестовая программа (программа No2), которая загружает библиотеки, используя только их местоположение и контракты.

Провести анализ двух типов использования библиотек.

Пользовательский ввод для обеих программ должен быть организован следующим образом:

1. Если пользователь вводит команду «0», то программа переключает одну реализацию контрактов на другую (необходимо только для программы No2). Можно реализовать лабораторную работу без данной функции, но максимальная оценка в этом случае будет «хорошо»;
2. «1 arg1 arg2 ... argN», где после «1» идут аргументы для первой функции, предусмотренной контрактами. После ввода команды происходит вызов первой функции, и на экране появляется результат её выполнения;
3. «2 arg1 arg2 ... argM», где после «2» идут аргументы для второй функции, предусмотренной контрактами. После ввода команды происходит вызов второй функции, и на экране появляется результат её выполнения

N	Описание	Сигнатура	Реализация 1	Реализация 2
2	Подсчёт количества простых чисел на отрезке [A, B] (A, B натуральные)	Int PrimeCount(int A, int B)	Наивный алгоритм. Проверить делимость текущего числа на все предыдущие числа.	Решето Эратосфена
8	Отсортировать целочисленный массив	Int * Sort(int * array, int size)	Пузырьковая сортировка	Сортировка Хоара

Общий метод и алгоритм решения

Для реализации поставленной задачи необходимо:

1. Изучить работу с библиотеками.
2. Реализовать две библиотеки согласно заданию.
3. Реализовать две программы (для работы с динамическими и статическими библиотеками).

Исходный код

// dynamic.c: Динамическая загрузка библиотек

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <dlfcn.h>
```

```
typedef int (*PrimeCountFunc)(int, int);
```

```
typedef void (*SortFunc)(int *, int);
```

```
int main() {
```

```
    void *lib1_handle;
```

```

void *lib2_handle;

PrimeCountFunc PrimeCount;

SortFunc Sort;

char *error;


lib1_handle = dlopen("./build/lib1.so", RTLD_LAZY);
lib2_handle = dlopen("./build/lib2.so", RTLD_LAZY);
if (!lib1_handle || !lib2_handle) {
    fprintf(stderr, "Error loading library: %s\n", dlerror());
    exit(1);
}


printf("select the library you want to use \n 1:lib1(quicksort and sieve of
eratosthenes) \n 2:lib2(bubble sort and naive prime counting) \n");

void *lib_handle;

int var;

scanf("%d", &var);

if (var == 1) lib_handle = lib1_handle;
else lib_handle = lib2_handle;


PrimeCount = (PrimeCountFunc)dlsym(lib_handle, "PrimeCount");
Sort = (SortFunc)dlsym(lib_handle, "Sort");


if ((error = dlerror()) != NULL) {
    fprintf(stderr, "Error loading symbols: %s\n", error);
    exit(1);
}


int choice;

```

```

printf("1: Count primes\n2: Sort array\nChoose option: ");
scanf("%d", &choice);

if (choice == 1) {
    int A, B;
    printf("Enter range A and B: ");
    scanf("%d %d", &A, &B);
    printf("Prime count: %d\n", PrimeCount(A, B));
} else if (choice == 2) {
    int size;
    printf("Enter array size: ");
    scanf("%d", &size);
    int *array = (int *)malloc(size * sizeof(int));
    printf("Enter array elements: ");
    for (int i = 0; i < size; i++) scanf("%d", &array[i]);
    Sort(array, size);
    printf("Sorted array: ");
    for (int i = 0; i < size; i++) printf("%d ", array[i]);
    printf("\n");
    free(array);
}

dlclose(lib_handle);
return 0;
}

// static.c: Линковка на этапе компиляции
#include <stdio.h>
#include <stdlib.h>

```

```

extern int PrimeCount(int A, int B);
extern void Sort(int *array, int size);

int main() {
    int choice;
    printf("1: Count primes\n2: Sort array\nChoose option: ");
    scanf("%d", &choice);

    if (choice == 1) {
        int A, B;
        printf("Enter range A and B: ");
        scanf("%d %d", &A, &B);
        printf("Prime count: %d\n", PrimeCount(A, B));
    } else if (choice == 2) {
        int size;
        printf("Enter array size: ");
        scanf("%d", &size);
        int *array = (int *)malloc(size * sizeof(int));
        printf("Enter array elements: ");
        for (int i = 0; i < size; i++) scanf("%d", &array[i]);
        Sort(array, size);
        printf("Sorted array: ");
        for (int i = 0; i < size; i++) printf("%d ", array[i]);
        printf("\n");
        free(array);
    }

    return 0;
}

```

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
```

```
// Подсчет количества простых чисел наивным методом
```

```
int PrimeCount(int A, int B) {
    int count = 0;
    for (int i = A; i <= B; i++) {
        if (i < 2) continue;
        bool is_prime = true;
        for (int j = 2; j * j <= i; j++) {
            if (i % j == 0) {
                is_prime = false;
                break;
            }
        }
        if (is_prime) count++;
    }
    return count;
}
```

```
// Функция для сортировки пузырьком
```

```
void bubbleSort(int *arr, int n) {
    for (int i = 0; i < n - 1; ++i) {
        // Флаг для оптимизации
        int swapped = 0;
        for (int j = 0; j < n - i - 1; ++j) {
            if (arr[j] > arr[j + 1]) {
                // Обмен элементов
```



```

        int temp = arr[j];
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
        swapped = 1;
    }
}
// Если не было обменов, массив уже отсортирован
if (!swapped) {
    break;
}
}
}

```

```

void Sort(int *array, int size) {
    bubbleSort(array, size);
}

```

```

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

```

```

int PrimeCount(int A, int B) {
    if (B < 2) return 0;
    bool *is_prime = (bool *)malloc((B + 1) * sizeof(bool));
    for (int i = 0; i <= B; i++) is_prime[i] = true;
    is_prime[0] = is_prime[1] = false;

    for (int i = 2; i * i <= B; i++) {
        if (is_prime[i]) {
            for (int j = i * i; j <= B; j += i) {

```

```

        is_prime[j] = false;
    }
}
}
int count = 0;
for (int i=A; i<=B; ++i){
    if (is_prime[i]) count++;
}
free(is_prime);
return count;
}

```

// Быстрая сортировка (сортировка Хоара)

```

void quicksort(int *array, int low, int high) {
    if (low < high) {
        int pivot = array[high];
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (array[j] <= pivot) {
                i++;
                int temp = array[i];
                array[i] = array[j];
                array[j] = temp;
            }
        }
        int temp = array[i + 1];
        array[i + 1] = array[high];
        array[high] = temp;
    }
}

```

```

    int pi = i + 1;
    quicksort(array, low, pi - 1);
    quicksort(array, pi + 1, high);
}
}

```

```

void Sort(int *array, int size) {
    quicksort(array, 0, size - 1);
}

```

Сборка программы

```

rai@rai-laptop:~/cods/os/lab4$ make
mkdir -p build
gcc -fPIC -Wall -shared src/lib1.c -o build/lib1.so
mkdir -p build
gcc -fPIC -Wall -shared src/lib2.c -o build/lib2.so
mkdir -p build
gcc src/static.c -o build/static -Lbuild -l1 -l2
mkdir -p build
gcc src/dynamic.c -o build/dynamic -ldl
rai@rai-laptop:~/cods/os/lab4$ make build/run_static
echo "#!/bin/bash" > build/run_static
echo "export LD_LIBRARY_PATH=build:\D_LIBRARY_PATH" >> build/run_static
echo "build/static" >> build/run_static
chmod +x build/run_static

```

Демонстрация работы программы

```

rai@rai-laptop:~/cods/os/lab4$ build/dynamic
select the library you want to use
1:lib1(quicksort and sieve of eratosthenes)
2:lib2(bubble sort and naive prime counting)

```

2

1: Count primes

2: Sort array

Choose option: 2

Enter array size: 4

Enter array elements: 2 3 1 4

Sorted array: 1 2 3 4

Выводы

В ходе лабораторной работы я познакомился с созданием динамических библиотек в ОС Linux, а также с возможностью загружать эти библиотеки в ходе выполнения программы. Динамические библиотеки помогают уменьшить размер исполняемых файлов. Загрузка динамических библиотек во время выполнения также упрощает компиляцию. Однако также можно подключить библиотеку к программе на этапе линковки. Она все равно загрузится при выполнении, но теперь программа будет изначально знать что и где искать. Если библиотека находится не в стандартной для динамических библиотек директории, необходимо также сообщить линкеру, чтобы тот передал необходимый путь в исполняемый файл. При помощи библиотек мы можем писать более сложные вещи, которые используют простые функции, структуры и т.п., написанные ранее и сохраненные в различных библиотеках.